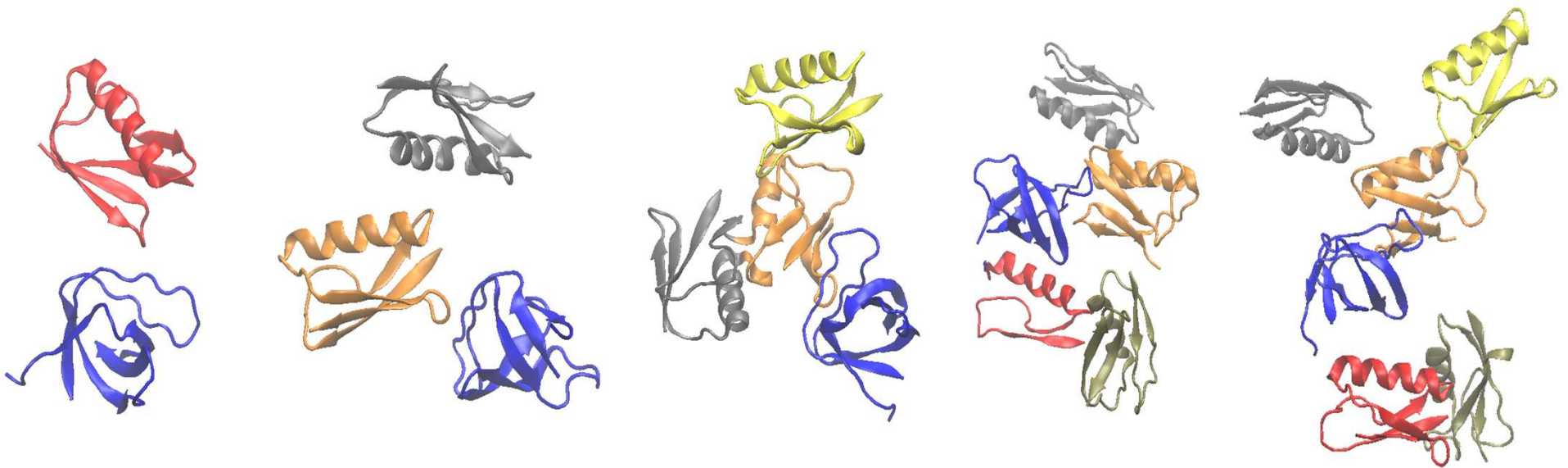


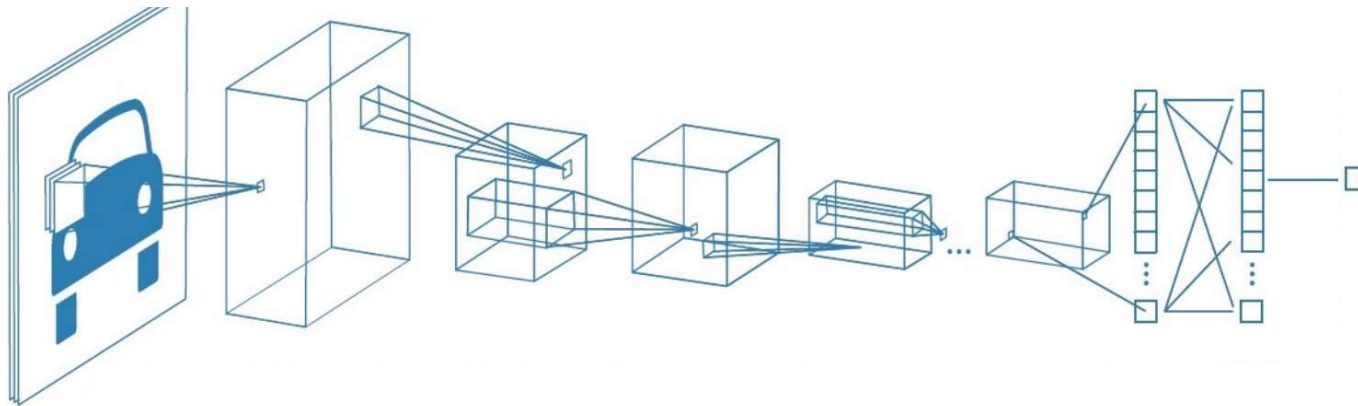
Do larger clusters of proteins diffuse through solution slower than smaller clusters?



Spencer Wozniak

Model

- To answer our question, we will design a model that:
 - Reads raw data describing a protein system
 - Transforms the data to a CSV file (readable by neural network)
 - Inputs data to trained neural network
 - Predicts the translational diffusion of a protein system



Computational techniques

- We need to generate data that will be used by the neural network to make a prediction.
 - For this neural network, our input data will consist of distances between alpha-carbon atoms and the system's center of mass.
 - This input data is somewhat arbitrary and other input data could be used, but this data is able to accurately describe the protein system.
- We start with a single file that represents a protein system.
 - The standard for storing protein information is the PDB file which looks something like this:

ATOM	1	N	MET	1	41.405	49.556	58.644	1.00	0.00	PSH3
ATOM	2	HT1	MET	1	41.158	48.570	58.410	1.00	0.00	PSH3
ATOM	3	HT2	MET	1	40.750	50.150	58.072	1.00	0.00	PSH3
ATOM	4	HT3	MET	1	41.174	49.725	59.647	1.00	0.00	PSH3
ATOM	5	CA	MET	1	42.796	49.808	58.289	1.00	0.00	PSH3
ATOM	6	HA	MET	1	43.116	50.616	58.932	1.00	0.00	PSH3
ATOM	7	CB	MET	1	43.651	48.575	58.606	1.00	0.00	PSH3
ATOM	8	HB1	MET	1	43.379	47.722	57.940	1.00	0.00	PSH3

```

# Defining 'Protein' class

class Protein:

    def __init__(self, pdb): # Constructor of 'Protein' class
        self.pdb = pdb # STRING: The pdb file a given 'Protein' object corresponds to
        self.atoms = self.readatoms() # LIST: List of dictionaries describing the important features of each atom
        self.calist = self.determine_ca() # LIST: List of indices of alpha carbon atoms in 'self.atoms'
        self.cofm = self.calc_cofm() # FLOAT: 'Protein' object's center of mass, as calculated in 'self.calc_cofm()'

    def readatoms(self): # returns List of dictionaries

        atom_list = [] # Initialize 'atom_list'

        for line in open(self.pdb): # Iterates over each line in the pdb file
            if line.split(' ')[0] == 'ATOM': # Given that the line holds ATOM information
                new = line.split(' ') # Splits string of each line into a list
                new = [i for i in new if i] # Removes all empty strings in 'new'

                # Reading a file this way is tricky because different lists may be made for each line based on formatting
                try:
                    int(new[5]) # If we can convert 'new[5]' to int, then
                    del new[4] # Then we move all elements after 'new[4]' back one index in the list
                except ValueError:
                    pass
                # Basically, this bit of code ensures we get the same input, regardless of formatting

                # creates dictionary for each atom
                atom_info = {'Element': new[2], # Reads type of atom
                             'X': float(new[5]), # X coordinate
                             'Y': float(new[6]), # Y coordinate
                             'Z': float(new[7])} # Z coordinate
                atom_list.append(atom_info) # Append dictionary to 'atom_list'

        return atom_list # Returns 'atom_list'

    def determine_ca(self): # returns List of indices of ca atoms in 'self.atoms'

        ca_list = [] # Initialize 'ca_list'

        for n in range(len(self.atoms)): # For all "atoms" in 'self.atoms'
            if self.atoms[n]['Element'] == 'CA': # If the "atom" is an alpha-carbon
                ca_list.append(n) # Append the index of that atom to 'ca_list'

        return ca_list # Returns 'ca_list'

    ## CA-COFM DIST METHODS ##

    def calc_cofm(self): # Calculates the center of mass using only alpha-carbon atoms
        # Alpha carbons are used so that the calculation is not as complicated (since all CA atoms are the same mass)

        # Initialize x,y,z lists
        x = []
        y = []
        z = []

        for i in self.calist: # For all indices contained in 'self.calist'
            x.append(self.atoms[i]['X'])
            y.append(self.atoms[i]['Y'])
            z.append(self.atoms[i]['Z'])

        # Center of mass coordinates are calculated as the average of all CA coordinates
        cofm = {'X': float(np.mean(x)), # X coordinate is the average of all CA X coordinates
                'Y': float(np.mean(y)), # Y coordinate is the average of all CA Y coordinates
                'Z': float(np.mean(z))} # Z coordinate is the average of all CA Z coordinates

        return cofm

    def all_ca_cofm_dist(self,x): # Calculates distances between all ca atoms and the center of mass

        # 'x' is the number of columns we will have after the translational diffusion coefficient in the .csv file
        # For this project, x will be set to 1000 (as described above)

        for i in range(x):
            try:
                # Prints distance, followed by a comma, given that there are i amino acids in the protein
                print(d3(self.atoms[self.calist[i]]['X'],self.atoms[self.calist[i]]['Y'],
                        self.atoms[self.calist[i]]['Z'],self.cofm['X'],self.cofm['Y'],self.cofm['Z']), end=',')
            except IndexError:
                # Prints zero, followed by a comma, if there are not i amino acids in the protein
                print(0, end=',')
        print()

```

Step 1

```
# Defining 'Protein' class

class Protein:

    def __init__(self, pdb): # Constructor of 'Protein' class
        self.pdb = pdb # STRING: The pdb file a given 'Protein' object corresponds to
        self.atoms = self.readatoms() # LIST: List of dictionaries describing the important features of each atom
        self.calist = self.determine_ca() # LIST: List of indices of alpha carbon atoms in 'self.atoms'
        self.cofm = self.calc_cofm() # FLOAT: 'Protein' object's center of mass, as calculated in 'self.calc_cofm()'

    def readatoms(self): # returns List of dictionaries

        atom_list = [] # Initialize 'atom_list'

        for line in open(self.pdb): # Iterates over each line in the pdb file
            if line.split(' ')[0] == 'ATOM': # Given that the line holds ATOM information
                new = line.split(' ') # Splits string of each line into a list
                new = [i for i in new if i] # Removes all empty strings in 'new'

                # Reading a file this way is tricky because different lists may be made for each line based on formatting
                try:
                    int(new[5]) # If we can convert 'new[5]' to int, then
                    del new[4] # Then we move all elements after 'new[4]' back one index in the list
                except ValueError:
                    pass
                # Basically, this bit of code ensures we get the same input, regardless of formatting

                # creates dictionary for each atom
                atom_info = {'Element': new[2], # Reads type of atom
                             'X': float(new[5]), # X coordinate
                             'Y': float(new[6]), # Y coordinate
                             'Z': float(new[7])} # Z coordinate
                atom_list.append(atom_info) # Append dictionary to 'atom_list'

        return atom_list # Returns 'atom_list'

    def determine_ca(self): # returns List of indices of ca atoms in 'self.atoms'

        ca_list = [] # Initialize 'ca_list'

        for n in range(len(self.atoms)): # For all "atoms" in 'self.atoms'
            if self.atoms[n]['Element'] == 'CA': # If the "atom" is an alpha-carbon
                ca_list.append(n) # Append the index of that atom to 'ca_list'

        return ca_list # Returns 'ca_list'

    ## CA-COFM DIST METHODS ##

    def calc_cofm(self): # Calculates the center of mass using only alpha-carbon atoms
        # Alpha carbons are used so that the calculation is not as complicated (since all CA atoms are the same mass)

        # Initialize x,y,z lists
        x = []
        y = []
        z = []

        for i in self.calist: # For all indices contained in 'self.calist'
            x.append(self.atoms[i]['X'])
            y.append(self.atoms[i]['Y'])
            z.append(self.atoms[i]['Z'])

        # Center of mass coordinates are calculated as the average of all CA coordinates
        cofm = {'X': float(np.mean(x)), # X coordinate is the average of all CA X coordinates
                 'Y': float(np.mean(y)), # Y coordinate is the average of all CA Y coordinates
                 'Z': float(np.mean(z))} # Z coordinate is the average of all CA Z coordinates

        return cofm

    def all_ca_cofm_dist(self,x): # Calculates distances between all ca atoms and the center of mass

        # 'x' is the number of columns we will have after the translational diffusion coefficient in the .csv file
        # For this project, x will be set to 1000 (as described above)

        for i in range(x):
            try:
                # Prints distance, followed by a comma, given that there are i amino acids in the protein
                print(d3(self.atoms[self.calist[i]]['X'],self.atoms[self.calist[i]]['Y'],
                        self.atoms[self.calist[i]]['Z'],self.cofm['X'],self.cofm['Y'],self.cofm['Z']), end=',')
            except IndexError:
                # Prints zero, followed by a comma, if there are not i amino acids in the protein
                print(0, end=',')
        print()
```

self.pdb

ATOM	1	N	MET	1	41.405	49.556	58.644	1.00	0.00	PSH3
ATOM	2	HT1	MET	1	41.158	48.570	58.410	1.00	0.00	PSH3
ATOM	3	HT2	MET	1	40.750	50.150	58.072	1.00	0.00	PSH3
ATOM	4	HT3	MET	1	41.174	49.725	59.647	1.00	0.00	PSH3
ATOM	5	CA	MET	1	42.796	49.808	58.289	1.00	0.00	PSH3
ATOM	6	HA	MET	1	43.116	50.616	58.932	1.00	0.00	PSH3
ATOM	7	CB	MET	1	43.651	48.575	58.606	1.00	0.00	PSH3
ATOM	8	HB1	MET	1	43.379	47.722	57.940	1.00	0.00	PSH3

```
for line in open(self.pdb): # Iterates over each line in the pdb file
    if line.split(' ')[0] == 'ATOM': # Given that the line holds ATOM information
        new = line.split(' ') # Splits string of each line into a list
        new = [i for i in new if i] # Removes all empty strings in 'new'
```

new

['ATOM', '1', 'N', 'MET', '1', '41.405', '49.556', '58.644', '1.00', '0.00', 'PSH3']
--

new ['ATOM', '1', 'N', 'MET', '1', '41.405', '49.556', '58.644', '1.00', '0.00', 'PSH3']

```
atom_info = {'Element': new[2], # Reads type of atom  
             'X': float(new[5]), # X coordinate  
             'Y': float(new[6]), # Y coordinate  
             'Z': float(new[7])} # Z coordinate
```

atom_info {'Element': 'N', 'X': '41.405', 'Y': '49.556', 'Z': '58.644'}

(NOTE: One dictionary will be stored for each atom, so there will be a list of 10,000+ dictionaries, stored in `self.atoms`)

```
self.atoms = self.readatoms()
```

Step 2

```
# Defining 'Protein' class

class Protein:

    def __init__(self, pdb): # Constructor of 'Protein' class
        self.pdb = pdb # STRING: The pdb file a given 'Protein' object corresponds to
        self.atoms = self.readatoms() # LIST: List of dictionaries describing the important features of each atom
        self.calist = self.determine_ca() # LIST: List of indices of alpha carbon atoms in 'self.atoms'
        self.cofm = self.calc_cofm() # FLOAT: 'Protein' object's center of mass, as calculated in 'self.calc_cofm()'

    def readatoms(self): # returns List of dictionaries

        atom_list = [] # Initialize 'atom_list'

        for line in open(self.pdb): # Iterates over each line in the pdb file
            if line.split(' ')[0] == 'ATOM': # Given that the line holds ATOM information
                new = line.split(' ') # Splits string of each line into a list
                new = [i for i in new if i] # Removes all empty strings in 'new'

                # Reading a file this way is tricky because different lists may be made for each line based on formatting
                try:
                    int(new[5]) # If we can convert 'new[5]' to int, then
                    del new[4] # Then we move all elements after 'new[4]' back one index in the list
                except ValueError:
                    pass
                # Basically, this bit of code ensures we get the same input, regardless of formatting

                # creates dictionary for each atom
                atom_info = {'Element': new[2], # Reads type of atom
                             'X': float(new[5]), # X coordinate
                             'Y': float(new[6]), # Y coordinate
                             'Z': float(new[7])} # Z coordinate
                atom_list.append(atom_info) # Append dictionary to 'atom_list'

        return atom_list # Returns 'atom_list'

    def determine_ca(self): # returns List of indices of ca atoms in 'self.atoms'

        ca_list = [] # Initialize 'ca_list'

        for n in range(len(self.atoms)): # For all "atoms" in 'self.atoms'
            if self.atoms[n]['Element'] == 'CA': # If the "atom" is an alpha-carbon
                ca_list.append(n) # Append the index of that atom to 'ca_list'

        return ca_list # Returns 'ca_list'

## CA-COFM DIST METHODS ##

    def calc_cofm(self): # Calculates the center of mass using only alpha-carbon atoms
        # Alpha carbons are used so that the calculation is not as complicated (since all CA atoms are the same mass)

        # Initialize x,y,z lists
        x = []
        y = []
        z = []

        for i in self.calist: # For all indices contained in 'self.calist'
            x.append(self.atoms[i]['X'])
            y.append(self.atoms[i]['Y'])
            z.append(self.atoms[i]['Z'])

        # Center of mass coordinates are calculated as the average of all CA coordinates
        cofm = {'X': float(np.mean(x)), # X coordinate is the average of all CA X coordinates
                 'Y': float(np.mean(y)), # Y coordinate is the average of all CA Y coordinates
                 'Z': float(np.mean(z))} # Z coordinate is the average of all CA Z coordinates

        return cofm

    def all_ca_cofm_dist(self,x): # Calculates distances between all ca atoms and the center of mass

        # 'x' is the number of columns we will have after the translational diffusion coefficient in the .csv file
        # For this project, x will be set to 1000 (as described above)

        for i in range(x):
            try:
                # Prints distance, followed by a comma, given that there are i amino acids in the protein
                print(d3(self.atoms[self.calist[i]]['X'],self.atoms[self.calist[i]]['Y'],
                        self.atoms[self.calist[i]]['Z'],self.cofm['X'],self.cofm['Y'],self.cofm['Z']), end=',')
            except IndexError:
                # Prints zero, followed by a comma, if there are not i amino acids in the protein
                print(0, end=',')
        print()
```

self.atoms

```
[{'Element': 'N', 'X': '41.405', 'Y': '49.556', 'Z': '58.644'}, (index 0)  
{ 'Element': 'HT1', 'X': '40.750', 'Y': '50.150', 'Z': '58.072'}, (index 1)  
{ 'Element': 'CA', 'X': '42.796', 'Y': '49.808', 'Z': '58.289'}, (index 2)  
...
```

```
def determine_ca(self): # returns list of indices of ca atoms in 'self.atoms'  
    ca_list = [] # Initialize 'ca_list'  
    for n in range(len(self.atoms)): # For all "atoms" in 'self.atoms'  
        if self.atoms[n]['Element'] == 'CA': # If the "atom" is an alpha-carbon  
            ca_list.append(n) # Append the index of that atom to 'ca_list'  
    return ca_list # Returns 'ca_list'
```

ca_list [2, 18, 47, 66, 98, 120, 151, 169, ... 2601, 2629] self.calist

Step 3

```
# Defining 'Protein' class
class Protein:

    def __init__(self, pdb): # Constructor of 'Protein' class
        self.pdb = pdb # STRING: The pdb file a given 'Protein' object corresponds to
        self.atoms = self.readatoms() # LIST: List of dictionaries describing the important features of each atom
        self.calist = self.determine_ca() # LIST: List of indices of alpha carbon atoms in 'self.atoms'
        self.cofm = self.calc_cofm() # FLOAT: 'Protein' object's center of mass, as calculated in 'self.calc_cofm()'

    def readatoms(self): # returns List of dictionaries

        atom_list = [] # Initialize 'atom_list'

        for line in open(self.pdb): # Iterates over each line in the pdb file
            if line.split(' ')[0] == 'ATOM': # Given that the line holds ATOM information
                new = line.split(' ') # Splits string of each line into a list
                new = [i for i in new if i] # Removes all empty strings in 'new'

                # Reading a file this way is tricky because different lists may be made for each line based on formatting
                try:
                    int(new[5]) # If we can convert 'new[5]' to int, then
                    del new[4] # Then we move all elements after 'new[4]' back one index in the list
                except ValueError:
                    pass
                # Basically, this bit of code ensures we get the same input, regardless of formatting

                # creates dictionary for each atom
                atom_info = {'Element': new[2], # Reads type of atom
                            'X': float(new[5]), # X coordinate
                            'Y': float(new[6]), # Y coordinate
                            'Z': float(new[7])} # Z coordinate
                atom_list.append(atom_info) # Append dictionary to 'atom_list'

        return atom_list # Returns 'atom_list'

    def determine_ca(self): # returns List of indices of ca atoms in 'self.atoms'

        ca_list = [] # Initialize 'ca_list'

        for n in range(len(self.atoms)): # For all "atoms" in 'self.atoms'
            if self.atoms[n]['Element'] == 'CA': # If the "atom" is an alpha-carbon
                ca_list.append(n) # Append the index of that atom to 'ca_list'

        return ca_list # Returns 'ca_list'

## CA-COFM DIST METHODS ##

    def calc_cofm(self): # Calculates the center of mass using only alpha-carbon atoms
        # Alpha carbons are used so that the calculation is not as complicated (since all CA atoms are the same mass)

        # Initialize x,y,z lists
        x = []
        y = []
        z = []

        for i in self.calist: # For all indices contained in 'self.calist'
            x.append(self.atoms[i]['X'])
            y.append(self.atoms[i]['Y'])
            z.append(self.atoms[i]['Z'])

        # Center of mass coordinates are calculated as the average of all CA coordinates
        cofm = {'X': float(np.mean(x)), # X coordinate is the average of all CA X coordinates
                'Y': float(np.mean(y)), # Y coordinate is the average of all CA Y coordinates
                'Z': float(np.mean(z))} # Z coordinate is the average of all CA Z coordinates

        return cofm

    def all_ca_cofm_dist(self,x): # Calculates distances between all ca atoms and the center of mass

        # 'x' is the number of columns we will have after the translational diffusion coefficient in the .csv file
        # For this project, x will be set to 1000 (as described above)

        for i in range(x):
            try:
                # Prints distance, followed by a comma, given that there are i amino acids in the protein
                print(d3(self.atoms[self.calist[i]]['X'],self.atoms[self.calist[i]]['Y'],
                        self.atoms[self.calist[i]]['Z'],self.cofm['X'],self.cofm['Y'],self.cofm['Z']), end=',')
            except IndexError:
                # Prints zero, followed by a comma, if there are not i amino acids in the protein
                print(0, end=',')
        print()
```

self.calist [2, 18, 47, 66, 98, 120, 151, 169, ... 2601, 2629]

self.atoms[i]

{'Element': 'CA', 'X': '42.796', 'Y': '49.808', 'Z': '58.289'},
{'Element': 'CA', 'X': '42.796', 'Y': '49.808', 'Z': '58.289'},
{'Element': 'CA', 'X': '42.796', 'Y': '49.808', 'Z': '58.289'},

```
# Initialize x,y,z lists  
x = []  
y = []  
z = []  
  
for i in self.calist: # For all indices contained in 'self.calist'  
    x.append(self.atoms[i]['X'])  
    y.append(self.atoms[i]['Y'])  
    z.append(self.atoms[i]['Z'])
```

x = [41.158, 40.750, 41.174, 42.796, ... 43.379]
y = [48.570, 50.150, 49.725, 49.808, ... 47.722]
z = [58.410, 58.072, 59.647, 58.289, ... 57.940]

```
x = [41.158, 40.750, 41.174, 42.796, ... 43.379]
y = [48.570, 50.150, 49.725, 49.808, ... 47.722]
z = [58.410, 58.072, 59.647, 58.289, ... 57.940]
```

```
# Center of mass coordinates are calculated as the average of all CA coordinates
cofm = {'X': float(np.mean(x)), # X coordinate is the average of all CA X coordinates
        'Y': float(np.mean(y)), # Y coordinate is the average of all CA Y coordinates
        'Z': float(np.mean(z))} # Z coordinate is the average of all CA Z coordinates

return cofm
```

```
self.cofm cofm = {'X': 42.541, 'Y': 50.031, 'Z': 59.315}
```

Step 4

```
# Defining 'Protein' class
class Protein:

    def __init__(self, pdb): # Constructor of 'Protein' class
        self.pdb = pdb # STRING: The pdb file a given 'Protein' object corresponds to
        self.atoms = self.readatoms() # LIST: List of dictionaries describing the important features of each atom
        self.calist = self.determine_ca() # LIST: List of indices of alpha carbon atoms in 'self.atoms'
        self.cofm = self.calc_cofm() # FLOAT: 'Protein' object's center of mass, as calculated in 'self.calc_cofm()'

    def readatoms(self): # returns List of dictionaries

        atom_list = [] # Initialize 'atom_list'

        for line in open(self.pdb): # Iterates over each line in the pdb file
            if line.split(' ')[0] == 'ATOM': # Given that the line holds ATOM information
                new = line.split(' ') # Splits string of each line into a list
                new = [i for i in new if i] # Removes all empty strings in 'new'

                # Reading a file this way is tricky because different lists may be made for each line based on formatting
                try:
                    int(new[5]) # If we can convert 'new[5]' to int, then
                    del new[4] # Then we move all elements after 'new[4]' back one index in the list
                except ValueError:
                    pass
                # Basically, this bit of code ensures we get the same input, regardless of formatting

                # creates dictionary for each atom
                atom_info = {'Element': new[2], # Reads type of atom
                             'X': float(new[5]), # X coordinate
                             'Y': float(new[6]), # Y coordinate
                             'Z': float(new[7])} # Z coordinate
                atom_list.append(atom_info) # Append dictionary to 'atom_list'

        return atom_list # Returns 'atom_list'

    def determine_ca(self): # returns List of indices of ca atoms in 'self.atoms'

        ca_list = [] # Initialize 'ca_list'

        for n in range(len(self.atoms)): # For all "atoms" in 'self.atoms'
            if self.atoms[n]['Element'] == 'CA': # If the "atom" is an alpha-carbon
                ca_list.append(n) # Append the index of that atom to 'ca_list'

        return ca_list # Returns 'ca_list'

    ## CA-COFM DIST METHODS ##

    def calc_cofm(self): # Calculates the center of mass using only alpha-carbon atoms
        # Alpha carbons are used so that the calculation is not as complicated (since all CA atoms are the same mass)

        # Initialize x,y,z lists
        x = []
        y = []
        z = []

        for i in self.calist: # For all indices contained in 'self.calist'
            x.append(self.atoms[i]['X'])
            y.append(self.atoms[i]['Y'])
            z.append(self.atoms[i]['Z'])

        # Center of mass coordinates are calculated as the average of all CA coordinates
        cofm = {'X': float(np.mean(x)), # X coordinate is the average of all CA X coordinates
                'Y': float(np.mean(y)), # Y coordinate is the average of all CA Y coordinates
                'Z': float(np.mean(z))} # Z coordinate is the average of all CA Z coordinates

        return cofm

    def all_ca_cofm_dist(self,x): # Calculates distances between all ca atoms and the center of mass

        # 'x' is the number of columns we will have after the translational diffusion coefficient in the .csv file
        # For this project, x will be set to 1000 (as described above)

        for i in range(x):
            try:
                # Prints distance, followed by a comma, given that there are i amino acids in the protein
                print(d3(self.atoms[self.calist[i]]['X'],self.atoms[self.calist[i]]['Y'],
                        self.atoms[self.calist[i]]['Z'],self.cofm['X'],self.cofm['Y'],self.cofm['Z']), end=',')
            except IndexError:
                # Prints zero, followed by a comma, if there are not i amino acids in the protein
                print(0, end=',')
        print()
```

self.calist

[2, 18, 47, 66, 98, 120, 151, 169, ... 2601, 2629]

self.atoms

```
[{'Element': 'N', 'X': '41.405', 'Y': '49.556', 'Z': '58.644'},  
{ 'Element': 'HT1', 'X': '40.750', 'Y': '50.150', 'Z': '58.072'},  
{ 'Element': 'CA', 'X': '42.796', 'Y': '49.808', 'Z': '58.289'},
```

self.cofm

{'X': 42.541, 'Y': 50.031, 'Z': 59.315}

3-D distance formula

```
def d3(x1,y1,z1,x2,y2,z2):  
    return (((x2 - x1) ** 2) + ((y2 - y1) ** 2) + ((z2 - z1) ** 2)) ** 0.5
```

$$d = \sqrt{(x_1 - x_0)^2 + (y_1 - y_0)^2 + (z_1 - z_0)^2}$$

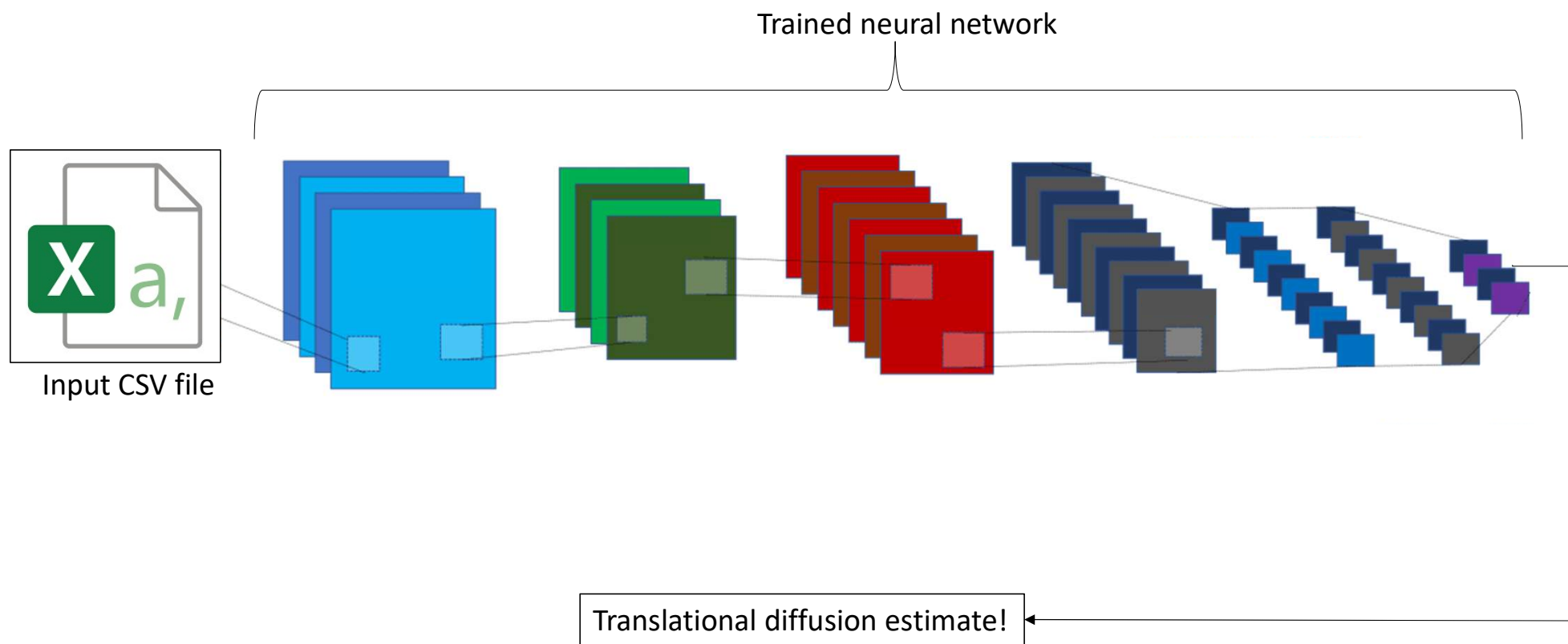
```
def all_ca_cofm_dist(self,x): # Calculates distances between all ca atoms and the center of mass  
  
    # 'x' is the number of columns we will have after the translational diffusion coefficient in the .csv file  
    # For this project, x will be set to 1000 (as described above)  
  
    for i in range(x):  
        try:  
            # Prints distance, followed by a comma, given that there are i amino acids in the protein  
            print(d3(self.atoms[self.calist[i]]['X'],self.atoms[self.calist[i]]['Y'],  
                    self.atoms[self.calist[i]]['Z'],self.cofm['X'],self.cofm['Y'],self.cofm['Z']), end=',')  
        except IndexError:  
            # Prints zero, followed by a comma, if there are not i amino acids in the protein  
            print(0, end=',')  
    print()
```

Single line of CSV file (1000 values separated by commas)

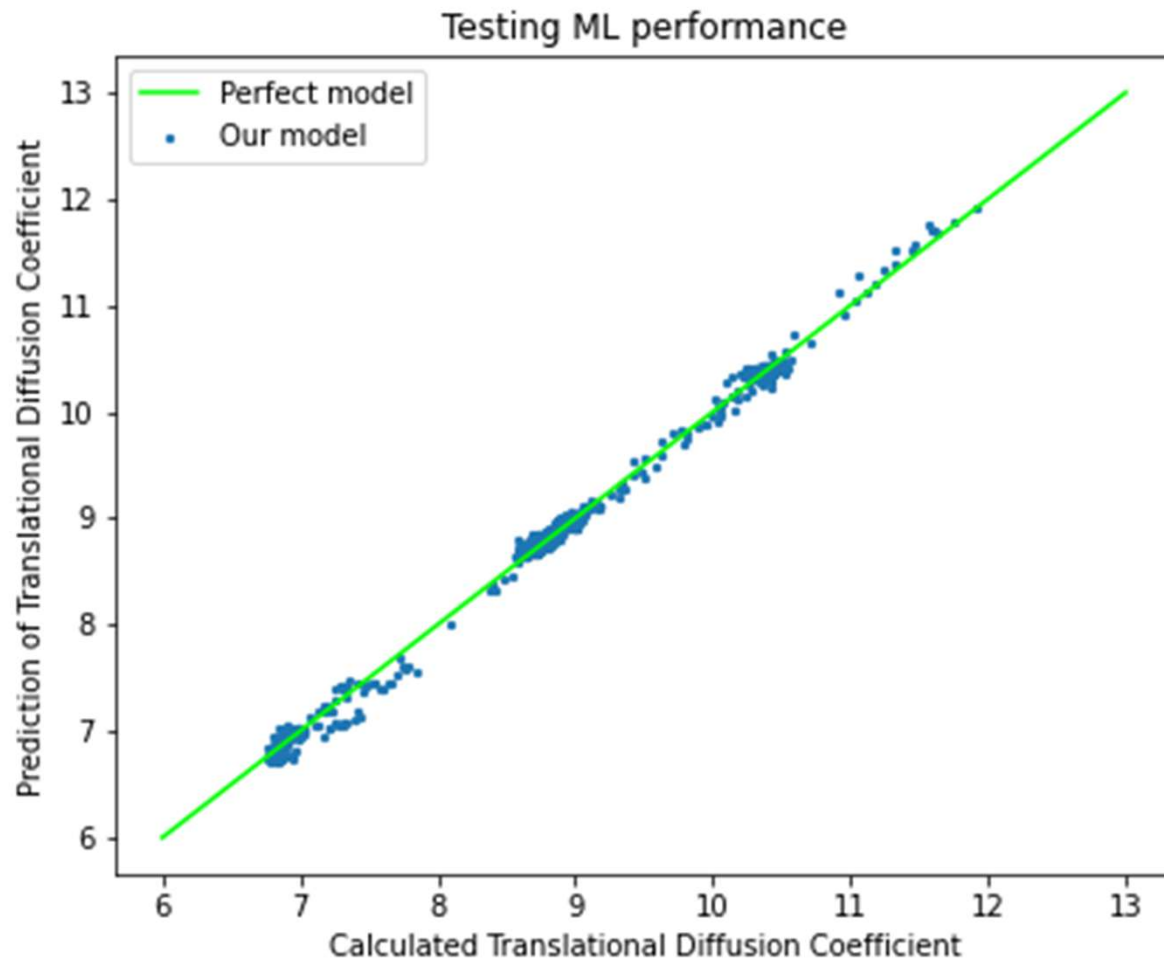
One line of the CSV file

[illegible]

(Repeat as many times as necessary)

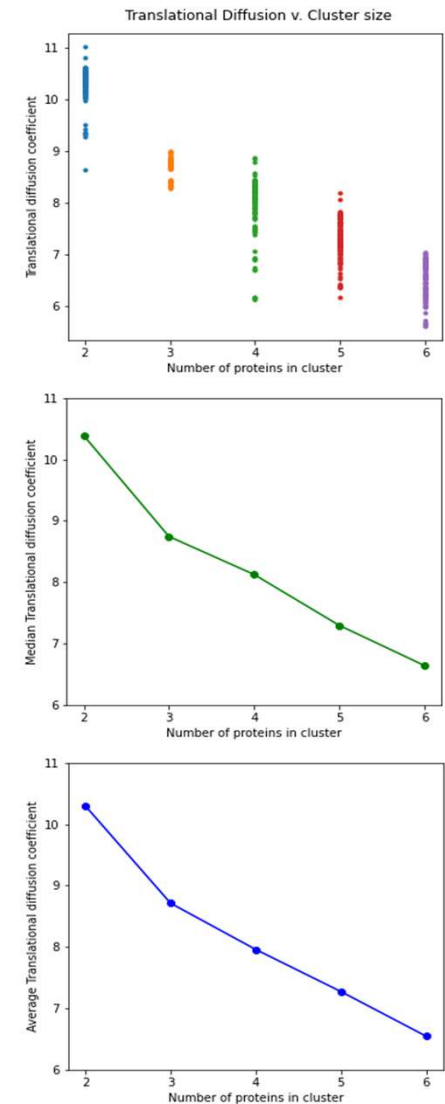


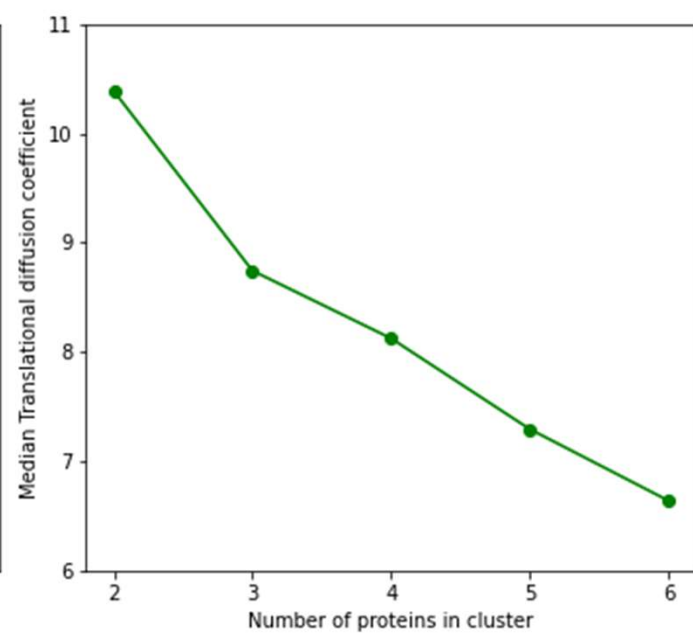
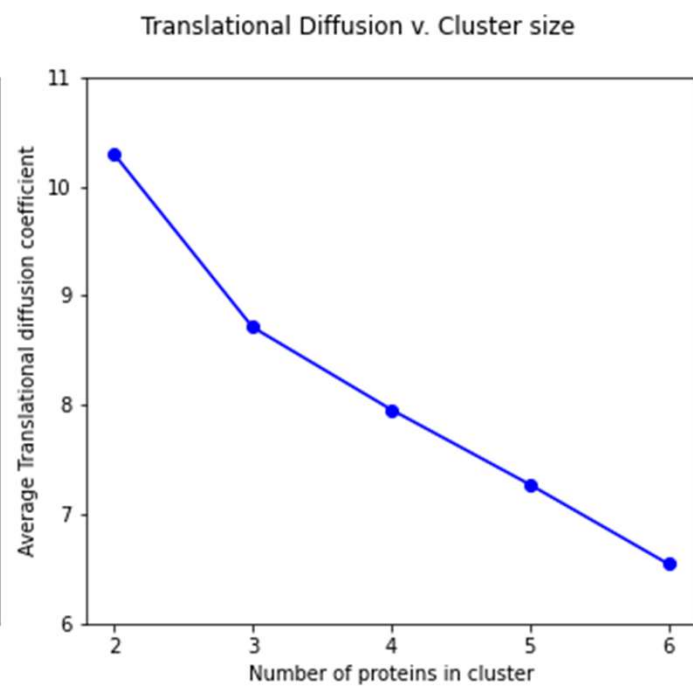
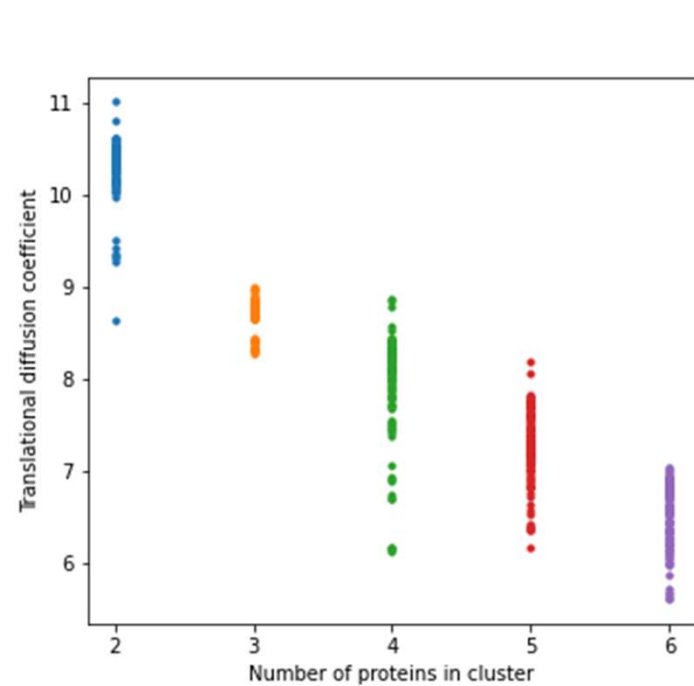
Results



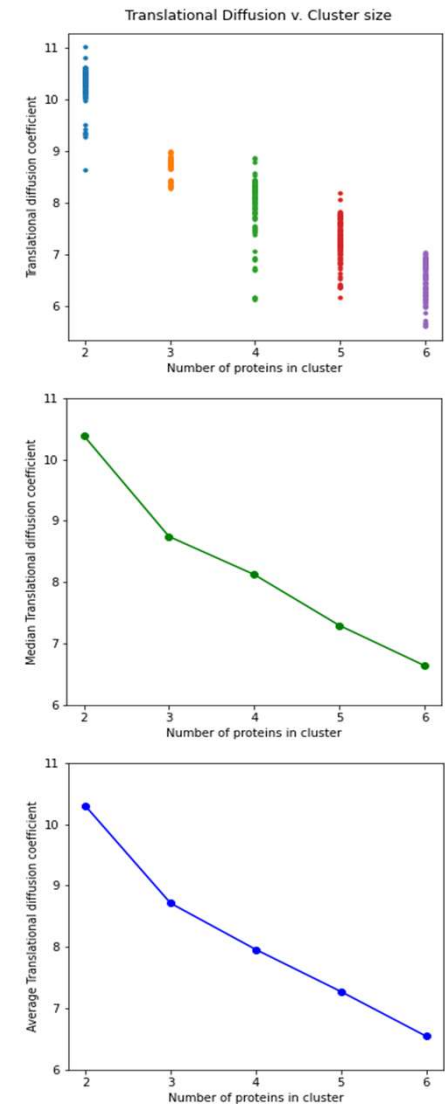
- The trained neural network was tested against 1,000 PDB files in which the translational diffusion was known.
- As seen on the graph, the model performed very well compared to what would be considered a perfect model.
- The mean percent error of our model was 0.006082003.

- We tested our question using clusters of two, three, four, five, and six proteins.
- 100 PDB files for each cluster size were tested with our trained model.
- Three graphs were created to describe the results:
 - Graph one plots all the predicted diffusions
 - Graph two plots the mean
 - Graph three plots the median





- Based on these graphs, generally, there is a negative correlation between translational diffusion and the number of proteins in a cluster.
- Therefore, it seems that larger protein clusters do diffuse through solution slower than smaller protein clusters.



Questions?